

Understanding AI Agents



Observe, Act, Achieve – The Agent Paradigm

Module 9: Agentic AI • Section 9.1

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover Today

1

What Are AI Agents?

The 'Observe, Act, Achieve' definition

2

Agent vs Chatbot vs Workflow

Where agents fit in the landscape

3

Perception, Reasoning, Action

The core agent loop

4

Architectures & Components

Model, Tools, Reasoning Loop

5

Categorization & Applications

Modality, interaction types, use cases

What Are AI Agents?

Defining a Generative AI Agent

A Generative AI Agent is defined as an **application** that tries to **achieve a goal** by **observing** the world and **acting** upon it using the **tools** it has at its disposal.

application

Software wrapping an LLM with logic

achieve a goal

Task-driven, not just response-driven

observing

Reads context, environment, tool outputs

acting

Calls tools, executes code, updates state

tools

External APIs, data, functions at disposal

Observe, Act, Achieve!

Agent vs Chatbot vs Workflow

Chatbot vs Workflow vs Agent

1

Chatbot

`prompt → LLM → answer`

Pattern: Single LLM call, no tools

Decisions: None

Example: Vanilla ChatGPT Q&A

Limit: Can't act on the world

2

Workflow

`prompt + context → LLM → answer`

Pattern: Predefined steps, fixed order

Decisions: Hardcoded by developer

Example: RAG pipeline (Days 1–4)

Limit: Can't deviate from script

3

Agent

`prompt → LLM → {tools, memory} →
action`

Pattern: Loop: Reason → Act → Observe

Decisions: LLM decides at runtime

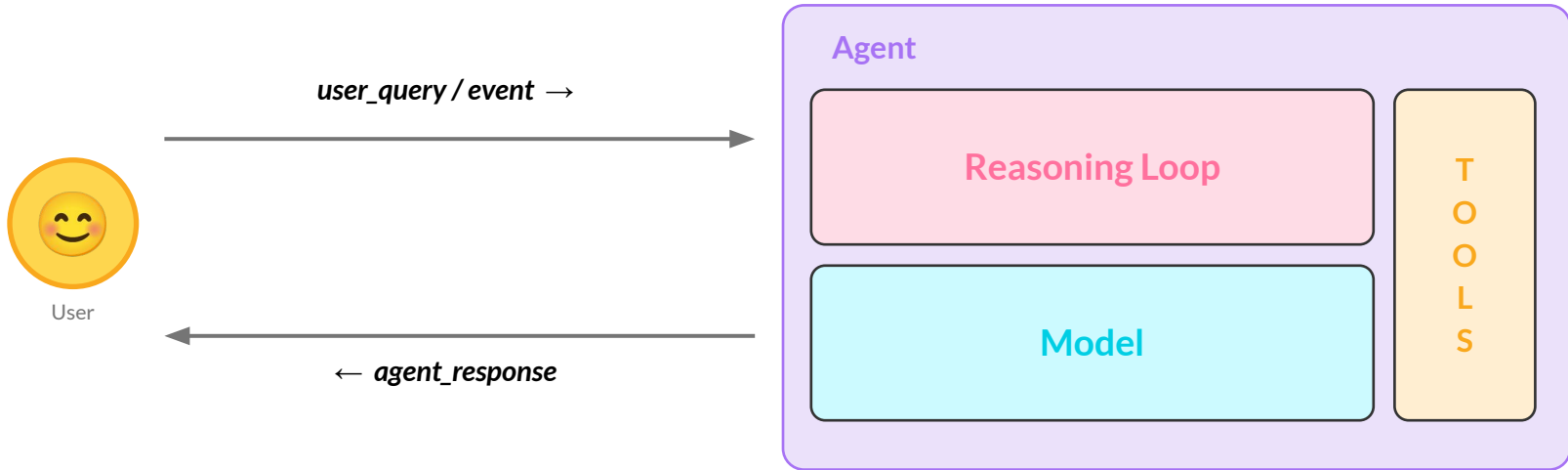
Example: Trip planner with tool calls

Strength: Adapts the path to the goal

Perception, Reasoning, Action

How Agent Interactions Begin

Trigger Types: Agents respond to *user_query* (a person asks something) or *event* (a system signals something happened).





The Reasoning Loop in Action

User: "I want to book a flight from Sydney to Melbourne."

Agent: "Here are some flights that might interest you..."

Question: "I want to book a flight..."

Thought: I should search for flights...

Action: Flights Tool

Action Input: Flights from Sydney to Melbourne

Observation: The flight tool returns many flight options

Thought: I should present these to the user

Final Answer: Here are some flights...

Tools

Flights

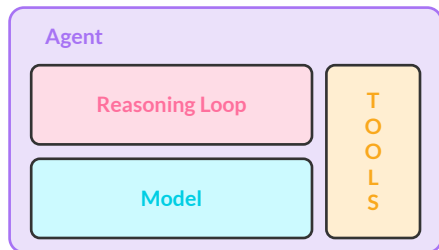
Search

Code

Calc

Foundational Components

Foundational Components of an Agent



Three pieces, one loop. Every agent — no matter how complex — boils down to a **Model** (the brain), **Tools** (the hands), and a **Reasoning Loop** (the orchestrator that decides what to do next).

1 Model

- Any Generative Language Model (large or small): GPT-4, Gemini, Claude
- Can be multimodal or fine-tuned: Gemini-Vision, DALL-E
- Training data plays a large role — prefer models trained with tool-use signatures

2 Tools

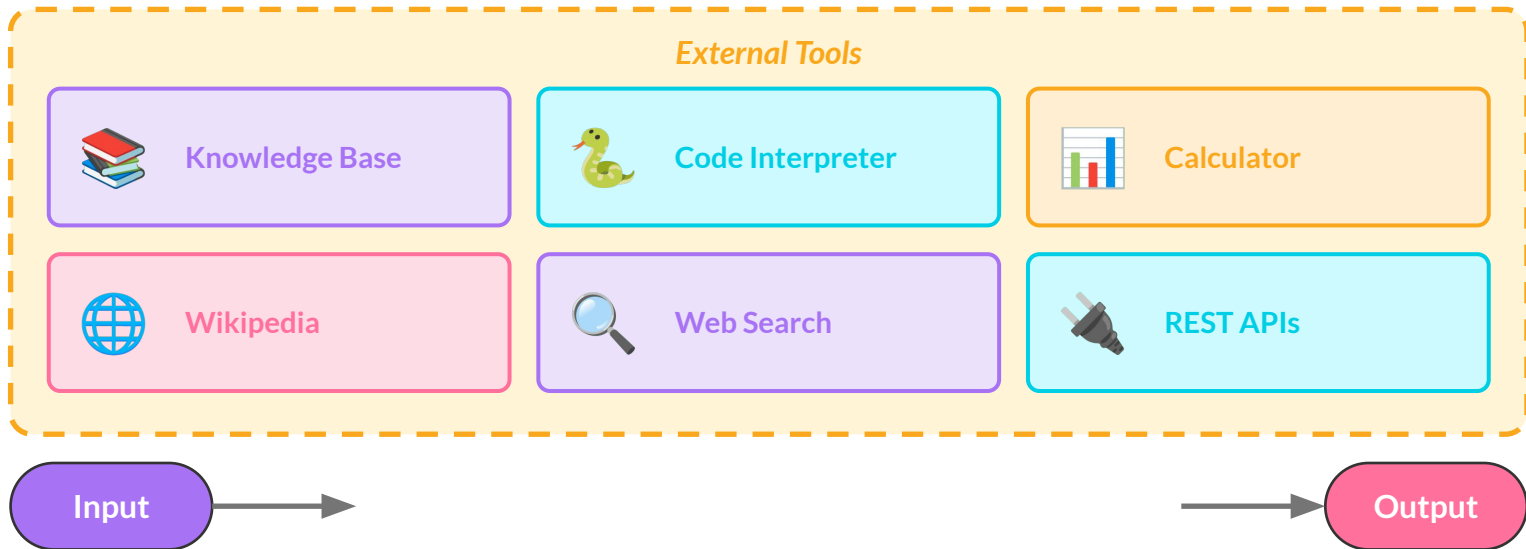
- Allow agents to interact with external data and services
- Three primary types:
 - Extensions
 - Functions
 - Data Stores

3 Reasoning Loop

- Iterative, introspective process — taking actions toward a goal
- Advanced prompt engineering frameworks:
 - Chain-of-Thought (CoT)
 - Tree-of-Thoughts (ToT)
 - ReAct, Directional Stimulus

External Tools: The Landscape

Beyond categories (Extensions/Functions/Data Stores), here are the **actual kinds of tools** agents call in the wild.



The Three Tool Types

Tools allow agents to interact with external data and services. There are three primary types — each with different control patterns and use cases.

1 Extensions

Agent ↔ Extension ↔ API

- Agent-side control — agent calls the API directly
- Bridges agent ↔ external API (e.g., Google Flights)
- Built using OpenAPI specs + frameworks like Flask
- Tells the agent: WHEN, HOW, and WHAT to expect

2 Functions

Agent → spec → Client → API

- Client-side control — agent returns a function spec
- Returns: {"function_call": {"name": "get_flights"...}}
- Useful when API is secure / privileged
- Division of labor: Agent decides, client acts

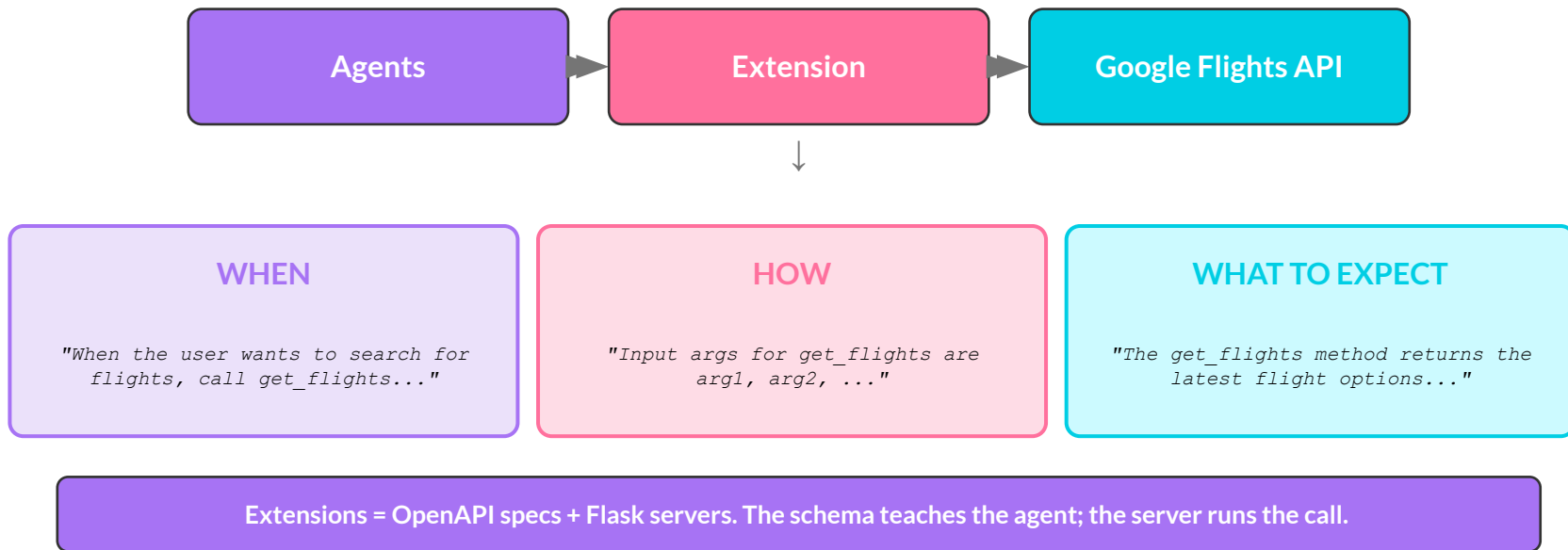
3 Data Stores

Query → Vector DB → Snippets

- Vector databases ground agents in fresh / private data
- Sources: Private Docs, Websites, Structured Data
- Returns ranked snippets: {id, url, snippet}
- This is RAG inside the agent — Days 1–4 territory

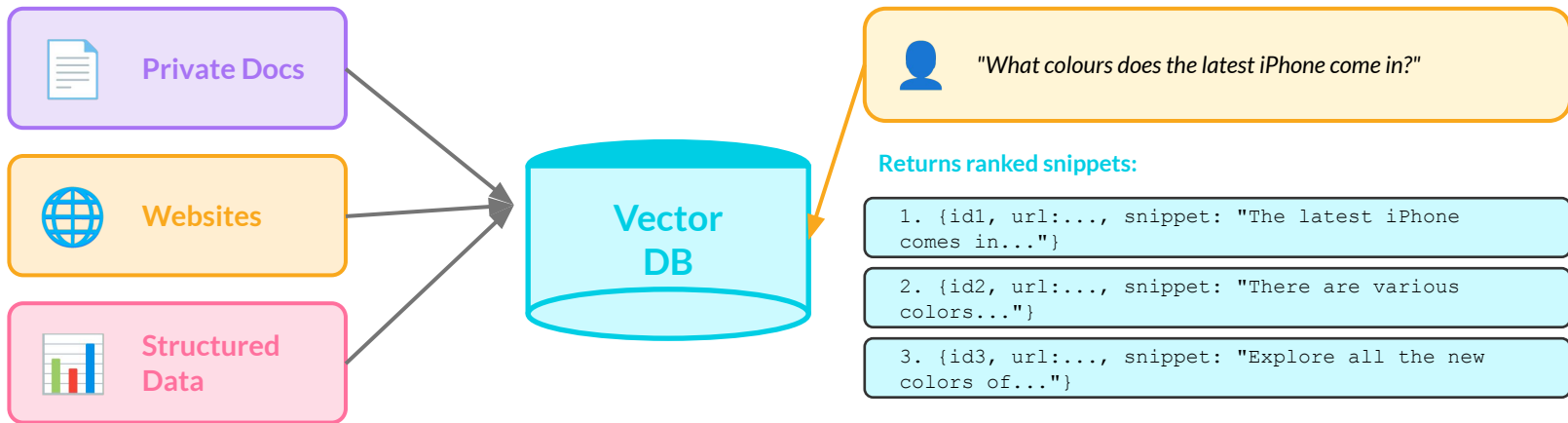
How Extensions Are Built

An Extension is what teaches the agent **WHEN** to call an API, **HOW** to call it, and **WHAT TO EXPECT** back.



Vector DB: Indexing & Retrieval Flow

Data Stores ingest from **many sources** into a single Vector DB, then return ranked snippets for any query — the live retrieval the agent calls.



Many sources in. One vector index. Ranked snippets out — that's RAG inside the agent.

GenAI Agent Categorization

Modality Based

- **LLM (Large Language Model)**, focus on language inputs
- **VLM (Vision Language Model)**, takes both Vision and Language inputs
- **VLA (Vision Language Action)**, combines vision and language to produce physical actions, often applied in robotics

Use Case Based

- **Gaming**, NPCs in games
- **Healthcare**, Diagnostic agents
- **Manufacturing**, VLA agents for assembly lines
- **Cybersecurity**, Adversarial testing, red teaming
- **Customer Support**, LLM-based agents
- **Software Development**, multi-agent frameworks for writing code

GenAI Agent Interaction Types

Conversational Agent

- Q&A, Chit Chat, World Knowledge interactions with humans
- User query triggered
- Fulfill user queries or transactions

Workflow Agent

- Limited or No Human Interaction
- Event driven triggers
- Fulfill queued tasks or chains of tasks

Conversational agents talk to people. Workflow agents talk to systems.

Sample Agent Architecture

Scenario: A user asks a road-trip planner agent – *"...road trip with my family..."* – and the agent orchestrates Functions, Data Stores, and Extensions to respond.

1

User input arrives at Custom UI: "...road trip with my family..."

2

UI forwards the query to the Agent

3

Agent assembles Prompt + Tool Definitions + Examples, sends to Model

4

Model decides which tool(s) to invoke – Function, Data Store, or Extension

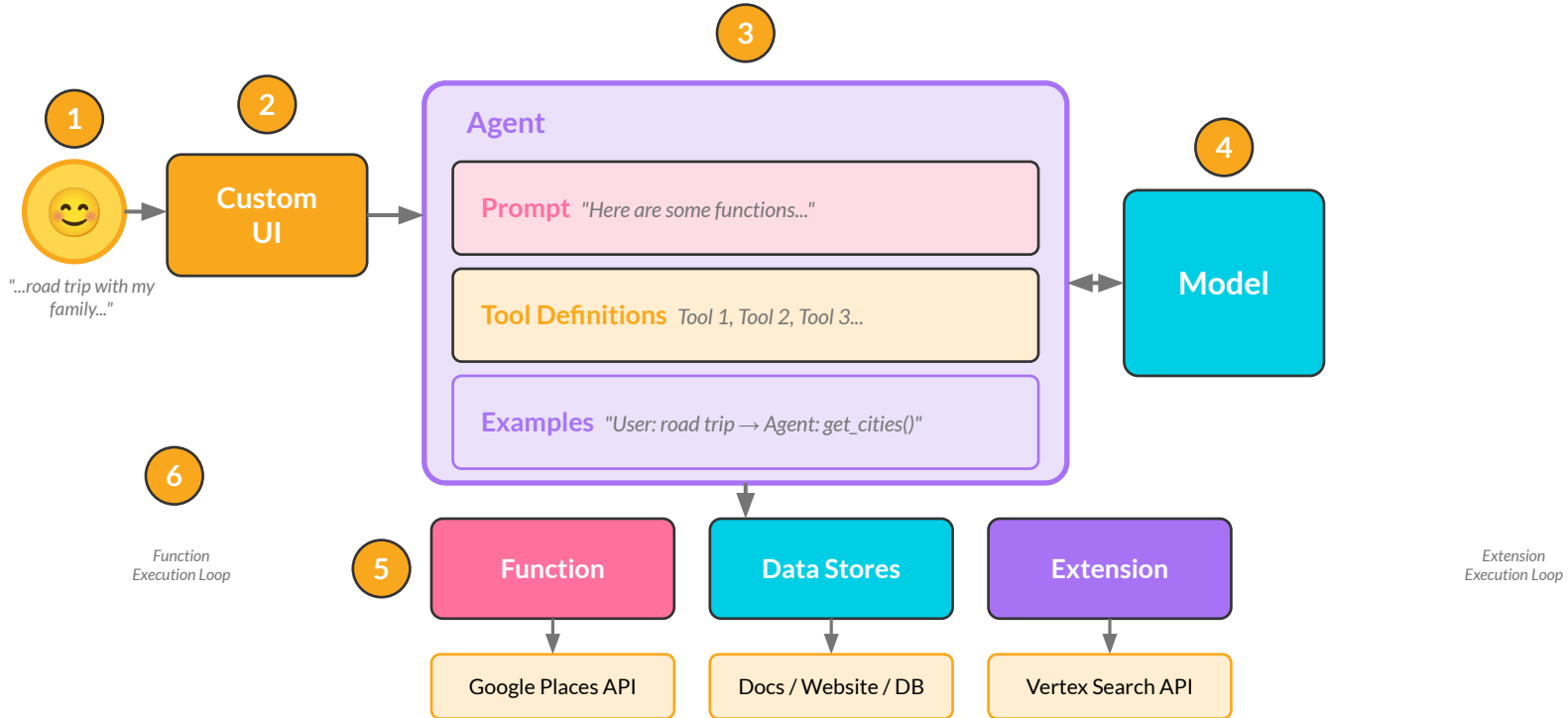
5

Tools execute: Function → Google Places API • Data Store → Docs/DB • Extension → Vertex Search

6

Function Execution Loop / Extension Execution Loop returns results to UI

Sample Agent Architecture



Let's Sum Up Shortly!

- 1 **Observe, Act, Achieve:** An app that pursues goals using tools
- 2 **LLM → RAG → Agent:** Each step adds capability: context, then reasoning + action
- 3 **Three Components:** Model + Tools + Reasoning Loop
- 4 **Three Tool Types:** Extensions, Functions, Data Stores
- 5 **Two Interaction Modes:** Conversational (user-triggered) + Workflow (event-triggered)

Next session: We'll go hands-on with Tools — building Functions and wiring them into a working agent.

Agentic Design Patterns



Four Patterns for Building Effective Agents

Module 9: Agentic AI • Section 9.2

Nunnari Academy

Generative AI & Agentic AI Program

The Four Agentic Design Patterns

Recall from 9.1: *An agent observes the world and acts upon it using tools to achieve a goal.* These four patterns are the recurring **blueprints** for building such agents — popularized by Andrew Ng (2024) and now powering most production agent systems.

1

Reflection

The agent critiques and improves its own output

2

Tool Use

The agent calls external tools to extend its capabilities

3

Planning

The agent decomposes a goal into a sequence of steps

4

Multi-Agent

Multiple specialized agents collaborate on a task

1

Reflection: Self-Evaluation & Improvement

Definition: The agent generates an output, then **critiques its own work** and revises it. A built-in feedback loop — the agent is its own reviewer.

1

GENERATE: Produce initial output (code, draft, answer)

2

REFLECT: "Is this correct? Complete? Clear?"

3

REVISE: Apply feedback, regenerate

🔄 **Loop until satisfied or max iterations reached**

Real Example: Code-Writing Agent

1. Agent writes a Python function
2. Agent runs linter + tests on its own code
3. Finds 2 bugs and a style issue
4. Rewrites the function
5. Re-runs checks → passes → done ✓

Builds on: Chain-of-Thought and ReAct — the reasoning frameworks from Module 9.1.

Reflection trades latency for quality — use when correctness matters more than speed.

2

Tool Use: Extending LLM Capabilities

Definition: The agent calls **external tools** — APIs, functions, databases, code interpreters — to do things the LLM alone cannot. The most foundational agentic pattern.

A

Extensions

- Agent-side control
- Direct API calls
- OpenAPI specs

B

Functions

- Client-side control
- Returns `function_call` JSON
- Used when API is privileged

C

Data Stores

- Vector DBs for grounding
- Returns ranked snippets
- RAG inside the agent

From Module 9.1 — the three tool types

Common tools: Web Search • Code Interpreter • Calculator • Knowledge Base • APIs • Databases

Without tools, an agent is just a chatbot. Tools turn reasoning into action.

3

Planning: Decomposition & Sequencing

Definition: The agent **breaks a complex goal into smaller sub-tasks** and figures out the order to execute them. Instead of one giant prompt, it builds a roadmap.

I-O

Direct

Input → LLM → Output

CoT

Chain-of-Thought

Linear reasoning steps

CoT-SC

Self-Consistency

Multiple chains → vote

ToT

Tree-of-Thoughts

Branching exploration

Reasoning frameworks from Module 9.1, ordered by sophistication →

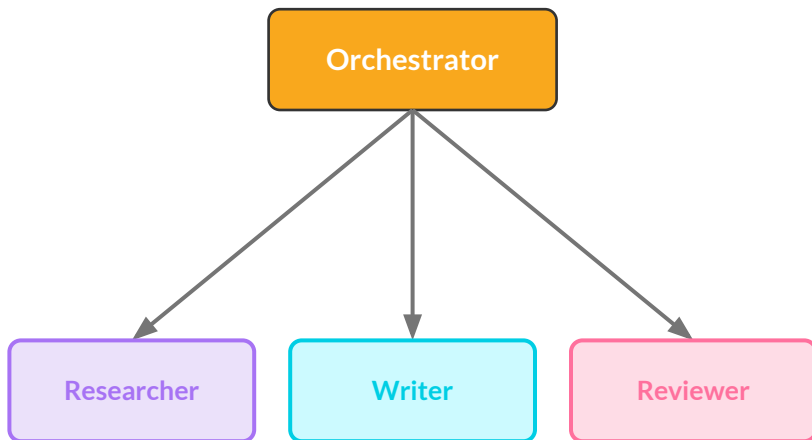
Goal: "Plan a 5-day trip to Bangalore for my family" → **Planning agent decomposes:** (1) Research family attractions (2) Check weather (3) Find hotels nearby (4) Build day-by-day itinerary (5) Estimate budget

Planning is what turns a one-shot answer into a multi-step strategy.

4

Multi-Agent: Collaborative Agent Systems

Definition: Multiple **specialized agents** work together — each with its own role, tools, and prompt — coordinated by an orchestrator. Division of labor for complex tasks.



Real Examples

Software Dev Team:

PM → Coder → Tester → Reviewer

Research Crew:

Searcher → Summarizer → Fact-checker

Customer Support:

Router → Specialist (billing/tech/sales)

Why? Each agent is simpler, easier to debug, and can use a smaller/cheaper model for its narrow role.

Multi-Agent = microservices for LLMs — small, focused, composable.

The Four Patterns at a Glance

	Reflection	Tool Use	Planning	Multi-Agent
Core idea	Self-critique	Call externals	Decompose	Collaborate
Key benefit	Higher quality	Real-world action	Tackle complexity	Specialization
Cost	More tokens	API latency	More steps	Coordination overhead
Best for	Code, writing, drafts	Anything beyond chat	Multi-step goals	Workflows with roles
Failure mode	Infinite loops	Tool errors	Bad plan = bad result	Agents talk past each other

Choosing the Right Pattern



Start with Tool Use: If your agent needs to do anything beyond pure text generation, tools come first



Add Reflection when correctness matters: Code generation, legal/medical drafts, anything you'd want a reviewer for



Add Planning for multi-step goals: When the path isn't obvious or branches based on intermediate results



Reach for Multi-Agent for complex workflows: When roles are clearly separable and one agent's prompt becomes unwieldy



Combine patterns: Production agents stack 2–4 patterns: e.g., Planning + Tool Use + Reflection in sequence

Start simple. Add patterns only when you hit the limits of the previous approach.

Let's Sum Up Shortly!

1

Reflection: Generate → Reflect → Revise. Self-critique loop for quality.

2

Tool Use: Extensions, Functions, Data Stores. Turns reasoning into action.

3

Planning: Decompose goals into sub-tasks. From I-O → CoT → ToT.

4

Multi-Agent: Specialized agents collaborate. Microservices for LLMs.

5

Stack Them: Real systems combine all four. Start simple, add as needed.

Next session: We'll implement these patterns hands-on with LangGraph.

Tool Calling & Function Calling



Connecting LLMs to the Real World

Module 9: Agentic AI • Section 9.3

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover Today

1

OpenAI Function Calling

The original spec: schemas, JSON, the loop

2

Anthropic Tool Use

Claude's tool API and how it differs

3

Building Custom Tools

Schema design, descriptions, parameters

4

Selection & Orchestration

How the LLM picks the right tool

5

Error Handling

Retries, fallbacks, graceful degradation

6

Security

Permissions, prompt injection, privilege boundaries

1

OpenAI Function Calling: The Concept

Definition: The model doesn't *call* the function — it returns a **structured JSON request** describing which function to call and with what arguments. **Your code executes it.**

1

YOU DEFINE: Tools as JSON schemas (passed in API call)

2

MODEL DECIDES: If/which to call → returns `tool_calls` block

3

YOU EXECUTE: Run the function and pass result back to model

Recall PDF p16:

Agent

Function

API

The LLM is the brain that decides; your code is the body that acts.

Defining a Function for OpenAI

Tool Schema (JSON):

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather
    for a specified city.",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string",
          "description": "City name"
        },
        "unit": {
          "type": "string",
          "enum": ["C", "F"]
        }
      },
      "required": ["city"]
    }
  }
}
```

Python Call:

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user",
         "content": "Weather in Bangalore?"}
    ],
    tools=[get_weather_schema],
    tool_choice="auto"
)

# Model returns:
# response.choices[0].message.tool_calls
# [{
#   "name": "get_weather",
#   "arguments": '{"city": "Bangalore"}'
# }]
```

Description fields matter — they're what the model reads to decide.

The Function Calling Loop

1 **User message** — sent to model with available tools

2 **Model returns tool_calls** — JSON spec, no execution yet

3 **Your code runs the function** — gets real result

4 **Send result back** — as a tool message with the tool_call_id

5 **Model generates final response** — using the tool result

Step 4 — Send result back:

```
messages.append({
    "role": "tool",
    "tool_call_id": call.id,
    "content": json.dumps(
        weather_result
    )
})

final = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tools
)

# final.choices[0].message.content
# → "It's 32°C and sunny
#    in Bangalore today."
```

Multi-tool, multi-turn, parallel calls — all built on this same loop.

2

Anthropic Tool Use with Claude

Definition: Claude's tool use API works on the **same principle** as OpenAI's, but with key differences in **schema format, stop reasons, and content blocks**. Designed to be more explicit and easier to parse.

Aspect	OpenAI	Anthropic (Claude)
Schema key	parameters	input_schema
Response field	tool_calls[]	content[].tool_use
Stop reason	finish_reason: tool_calls	stop_reason: tool_use
Result role	tool	user (with tool_result)

Same loop. Different field names. Both follow: spec → call → result → respond.

Defining a Tool for Claude

Tool Definition:

```
tools = [{
    "name": "get_weather",
    "description": "Get current weather
    for a specified city.",
    "input_schema": {
        "type": "object",
        "properties": {
            "city": {
                "type": "string",
                "description": "City name"
            },
            "unit": {
                "type": "string",
                "enum": ["C", "F"]
            }
        },
        "required": ["city"]
    }
}]
```

API Call & Response:

```
response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    tools=tools,
    messages=[{
        "role": "user",
        "content": "Weather in Mumbai?"
    }]
)

# response.content:
# [{
#     "type": "tool_use",
#     "id": "toolu_01A...",
#     "name": "get_weather",
#     "input": {"city": "Mumbai"}
# }]
```

Same mental model as OpenAI — just different field names.

Sending Tool Results Back to Claude

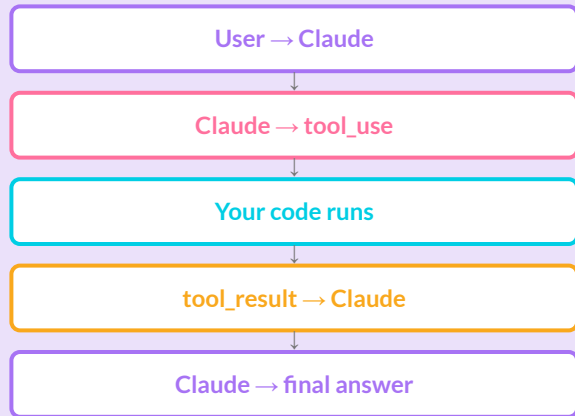
Key difference: Tool results go back as a **user** message containing a **tool_result** block, referenced by the **tool_use_id**.

```
messages.append({
    "role": "assistant",
    "content": response.content    # tool_use block
})

messages.append({
    "role": "user",
    "content": [{
        "type": "tool_result",
        "tool_use_id": "toolu_01A...",
        "content": "32°C, sunny"
    }]
})

final = client.messages.create(
    model="claude-opus-4-7",
    tools=tools,
    messages=messages
)
```

The Loop:



Claude treats tool results as 'user input' — keeping the conversation cleanly turn-based.

3

What Makes a Good Tool?



Clear, action-verb name: `get_weather`, `search_flights`, `send_email` (not `weather_thing`)



Detailed description: The model reads this to decide when to call. Be specific about when and why.



Typed parameters with descriptions: Each param gets a type, description, and required flag



Use enums for fixed choices: `unit: ['C', 'F']` is better than free-text `unit: string`



Idempotent where possible: Calling twice should be safe; agents retry on failures

✗ Bad: `"description": "Gets weather"`

✓ Good: `"Get current weather conditions for a specified city. Use when the user asks about temperature, rain, or weather forecasts."`

Building Tools the Easy Way: LangChain @tool

You've seen this before: In Day 4's RAG bootcamp and Day 5's LangGraph notebook, you used the `@tool` decorator to wrap Python functions into agent-callable tools. Same pattern, framework-agnostic.

```
from langchain_core.tools import tool

@tool
def search_flights(
    origin: str,
    destination: str,
    date: str
) -> str:
    """Search for flights between two
    cities on a given date.

    Args:
        origin: 3-letter airport code
        destination: 3-letter airport code
        date: Date in YYYY-MM-DD format
    """
    return f"Found 12 flights"

llm_with_tools = llm.bind_tools(
```

How it works:

- ✓ **Docstring** → tool description
- ✓ **Type hints** → JSON schema
- ✓ **Args section** → param descriptions
- ✓ **Zero boilerplate**
- ✓ **Framework-agnostic** — works with OpenAI, Anthropic, Ollama

One decorator. Works with OpenAI, Anthropic, Ollama — anywhere LangChain runs.

4

Tool Selection: How the LLM Decides

Selection happens in the model's reasoning step. Given the user query and the list of tool descriptions, the LLM decides: *which tool, with what arguments, or just answer directly?*

Recall PDF p8 – ReAct trace:

Question: "I want to book a flight from Sydney to Melbourne."

Thought: I should search for flights...

Action: Flights Tool ← tool selection

Action Input: {from: "SYD", to: "MEL"} ← argument extraction

Observation: 12 flight options returned

Final Answer: Here are some flights...

3 Selection Signals:

1

Tool descriptions

Most important; the model reads them every turn

2

User query semantics

Keywords, intent, entities

3

Conversation history

What's already been tried

Bad descriptions = bad selection. Spend more time on descriptions than on code.

Orchestrating Multiple Tools

When the agent has many tools, it may need to call several in sequence (or parallel) to complete one user request. This is **orchestration**.

Recall PDF p24 – Sample Architecture flow:

- 1 User asks: "Plan a road trip with my family"
- 2 Agent calls find_destinations (Function → Google Places API)
- 3 Agent calls lookup_weather (Extension → Weather API)
- 4 Agent calls search_docs (Data Store → Vector DB)
- 5 Agent synthesizes results across all three
- 6 Returns unified itinerary to user

Orchestration Patterns:

Sequential

Tool A's output → Tool B's input

Parallel

Call A, B, C at once, merge

Conditional

Call B only if A returns X

LangGraph

Models all 3 with explicit edges

Modern APIs (OpenAI, Anthropic) support parallel tool calls in one response.

5

Error Handling & Fallbacks



Tool throws an exception: Catch it. Return the error message as the tool result so the LLM can react.



Tool returns garbage: Validate output before passing back. Schema-check or sanity-check.



Wrong tool selected: Improve descriptions. Use `tool_choice="required"` or restrict tools per query.



Hallucinated arguments: Use strict JSON Schema validation. Reject and re-prompt with a clear error.



Infinite loop: Set a `max_iterations` cap (e.g., 10). Always.

Error-as-message pattern:

```
try:
    result = tool.invoke(args)
except Exception as e:
    result = (
        f"Error: {str(e)}. "
        "Try different arguments."
    )

messages.append({
    "role": "tool",
    "content": result
})

# Let the LLM decide
# what to do next
```

Treat errors as conversation, not exceptions. The LLM is surprisingly good at recovering.

6

Security: What to Worry About

Recall from Module 9.1: PDF p15's "Stubbing APIs & Division of Labour" — the *Super Secret API* sits behind a security boundary. The agent should never touch privileged systems directly.

1

Privilege Separation

Use Functions (client-side control) for anything privileged. The LLM proposes; your trusted code disposes. Never give the agent direct access to a Super Secret API.

2

Prompt Injection

Tool outputs may contain malicious instructions ("Ignore previous instructions and..."). Treat tool outputs as untrusted data, not commands.

3

Argument Validation

Validate every argument server-side. The LLM may hallucinate `delete_user(id="*")`. Type-check, range-check, allow-list.

4

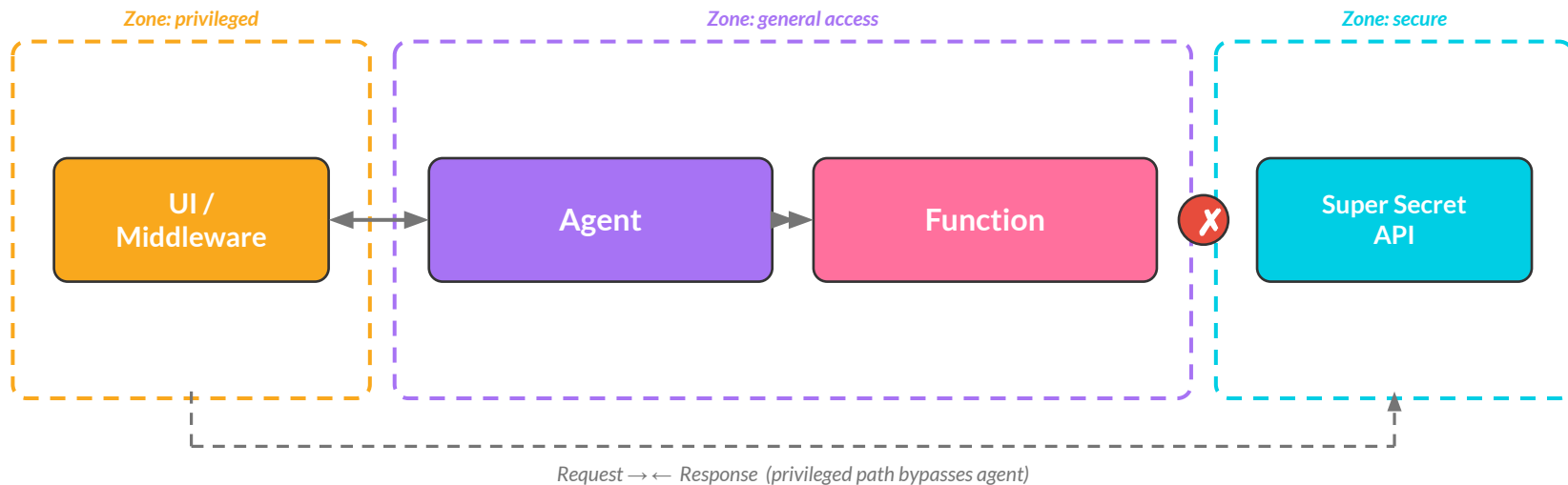
Audit & Rate Limit

Log every tool call. Rate-limit per user/session. Set spending caps if tools cost money (paid APIs, LLM tokens).

The agent is a user — give it the least privilege it needs to do the job.

Division of Labour: Security Boundaries

Recall PDF p15: Production agents live across **security zones**. The agent stays in 'general access' — never crosses into the privileged or secure zones directly.



The agent proposes; trusted code disposes. Privileged calls never touch the LLM.

Let's Sum Up Shortly!

1

Same Loop, Two Specs: OpenAI tool_calls and Anthropic tool_use follow the same model: spec → call → result → respond.

2

Descriptions = Selection: The LLM picks tools based on descriptions. Write them like docs for a junior dev.

3

@tool Saves Time: LangChain wraps Python functions into agent-callable tools with one decorator.

4

Errors Are Conversation: Pass error messages back as tool results. The LLM recovers.

5

Least Privilege: Functions over Extensions for privileged APIs. Validate. Audit. Rate-limit.

Next session: We'll build a working tool-calling agent end-to-end.

Building Agents from Scratch



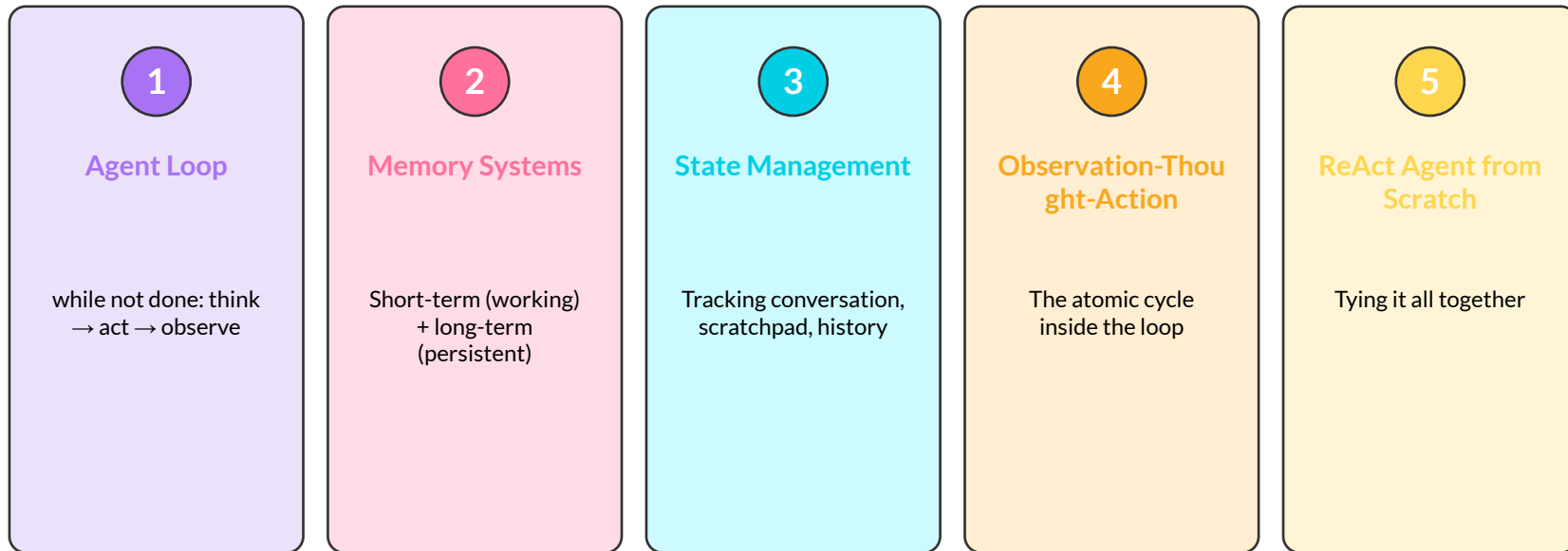
Hands-On – From Loop to Working ReAct Agent

Module 9: Agentic AI • Section 9.4

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Build Today



Each topic gets a code-first slide — syntax to take away, not full implementations.

1

The Agent Loop: The Skeleton

Recall PDF p5: "The reasoning loop is an iterative, introspective process aimed at taking actions toward a goal." In code, this is a **while** loop with three phases per iteration.

```
def agent_loop(query, tools, max_iters=10):
    messages = [{"role": "user", "content": query}]

    for i in range(max_iters):
        # 1. THINK - ask the LLM
        response = llm.invoke(messages, tools=tools)

        # 2. CHECK - done if no tool call
        if not response.tool_calls:
            return response.content

        # 3. ACT - execute each tool call
        for call in response.tool_calls:
            result = execute_tool(call.name, call.args)
            messages.append({"role": "tool",
                             "content": result})

    return "Max iterations reached."
```

The Loop (PDF p7):



Three lines of logic. Everything else is plumbing.

2

Memory Systems: Short-Term & Long-Term

Recall PDF p26 – the memory taxonomy: Sensory → Prompt • Short-Term (Working Memory) → Chat • Long-Term (Declarative + Procedural) → Data

Short-Term Memory

What: Current conversation + scratchpad

Where: In-memory list (Python messages = [])

Lifespan: This session only

```
messages = []
messages.append({
    "role": "user",
    "content": q})
messages.append({
    "role": "assistant",
    "content": a})
```

Long-Term Memory

What: Facts, preferences, past decisions

Where: Vector DB (Chroma/FAISS) or KV store

Lifespan: Persistent across runs

```
# Save
memory_store.add(
    text="User prefers concise",
    metadata={"user_id": "shivu"})

# Retrieve
ctx = memory_store.query(query, k=3)
```

Short-term = the conversation. Long-term = what the agent remembers about you.

3

State Management: What Lives Where

Recall PDF p24: the agent's state isn't just messages — it's **Prompt + Tool Definitions + Examples + Conversation + Scratchpad**. All of this needs to be tracked and passed forward each iteration.

```
from typing import TypedDict, List

class AgentState(TypedDict):
    # Static (set once)
    system_prompt: str
    tools: List[Tool]

    # Dynamic (updated each iteration)
    messages: List[dict] # conversation
    scratchpad: List[str] # agent's notes
    iteration: int # loop counter
    done: bool # exit flag
```

Why TypedDict?

- ✓ **Single source of truth**
for what the agent knows
- ✓ **Easy to serialize**
save / resume / debug
- ✓ **LangGraph uses this**
StateGraph(AgentState)
- ✓ **Each node reads + updates state**

State is the agent's working memory made explicit.

4

One Iteration: Thought, Action, Observation

This is the atomic unit of agent reasoning. Every loop iteration produces one Thought, one Action, one Observation. PDF p8's flight-booking trace is the canonical example.

PDF p8 – One iteration:

Thought

"I should search for flights..."

Action

Flights Tool + {from: SYD, to: MEL}

Observation

"The flight tool returns 12 options"

In code:

```
# THOUGHT – the LLM's reasoning
thought = response.content

# ACTION – the tool call it chose
action = response.tool_calls[0]

# OBSERVATION – the result we feed back
observation = execute_tool(action)
messages.append({
    "role": "tool",
    "content": observation
})
# Loop back → next Thought
```

Every agent step = 1 Thought + 1 Action + 1 Observation. Repeat.

5

ReAct Agent: The System Prompt

ReAct = Reasoning + Acting. The system prompt teaches the model the exact format from PDF p8: **Question → Thought → Action → Observation → ... → Final Answer.**

```
REACT_PROMPT = """You are a helpful agent. Use this exact format:
```

```
Question: the user's question
Thought: reason about what to do next
Action: the tool to call, one of [{tool_names}]
Action Input: the arguments to pass
Observation: the tool's result
... (Thought / Action / Observation can repeat)
Thought: I now know the final answer
Final Answer: the answer to the user
```

```
Available tools: {tool_descriptions}
Question: {user_query}
""" # Note: the format IS the algorithm
```

The format IS the algorithm. The model follows the structure you give it.

ReAct Agent: Parsing & Executing

The loop reads the LLM's text output, extracts **Action** and **Action Input**, runs the tool, appends **Observation**, and loops — until the model emits **Final Answer**.

```
def react_agent(query, tools, max_iters=10):
    prompt = REACT_PROMPT.format(
        tool_names=[t.name for t in tools],
        tool_descriptions=format_tools(tools),
        user_query=query)
    history = prompt

    for _ in range(max_iters):
        # Think
        output = llm.invoke(history, stop=["Observation:"])
        history += output
        # Done?
        if "Final Answer:" in output:
            return output.split("Final Answer:")[-1].strip()
        # Parse + execute → append Observation
        action = parse_action(output)
        action_input = parse_action_input(output)
        result = run_tool(action, action_input, tools)
        history += f"\nObservation: {result}\n"
```

Same loop as Slide 3 — just with text parsing instead of structured tool_calls.

Let's Sum Up Shortly!

1

Agent Loop: while not done: think → act → observe. Cap iterations.

2

Memory: Short-term = messages list. Long-term = vector DB.

3

State: TypedDict makes the agent's working memory explicit and serializable.

4

OTA Cycle: Every iteration = 1 Thought + 1 Action + 1 Observation.

5

ReAct from Scratch: Prompt teaches the format; loop parses + executes.

You've now built — by hand — what LangGraph's `create_react_agent` does in one line.

Thank You!



Nunnari Academy

Generative AI & Agentic AI Program